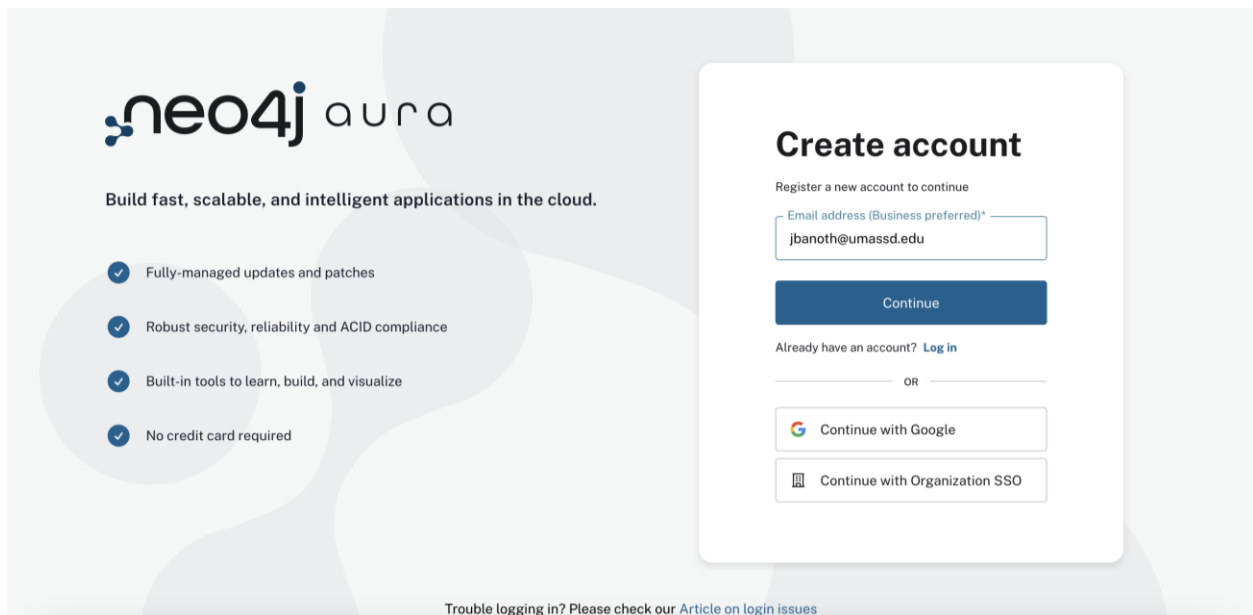


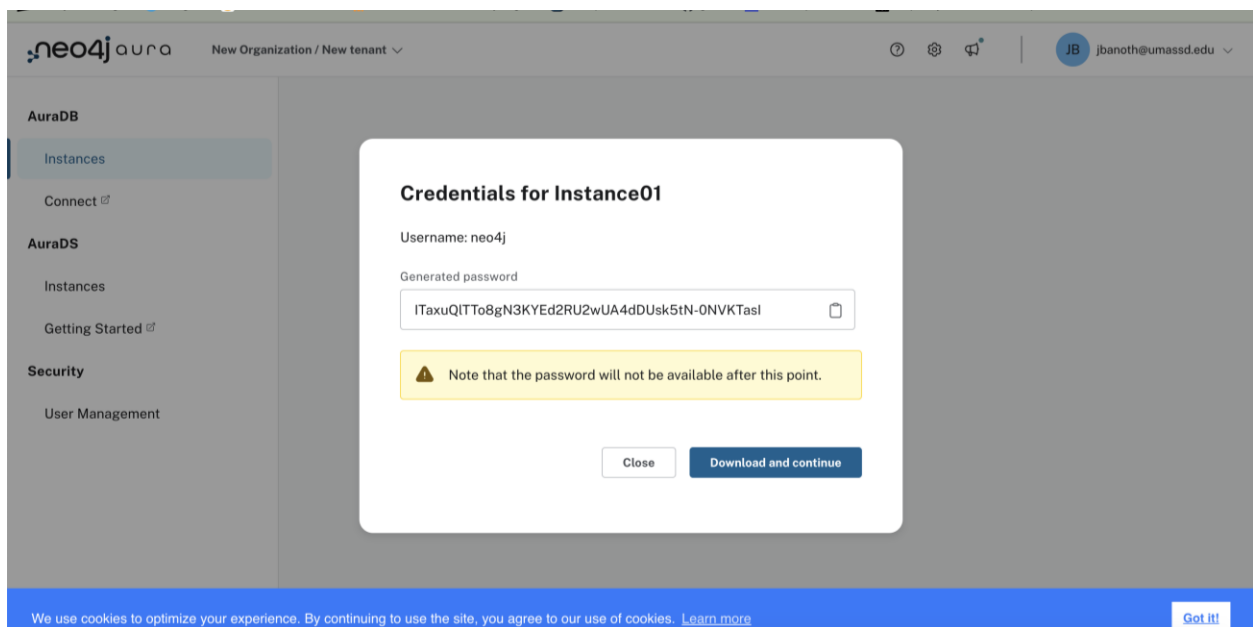
Jeevan Kumar Banoth – 02105145
CIS 552: Database Design
Homework – 8:

Create an Account in Neo4j AURA:



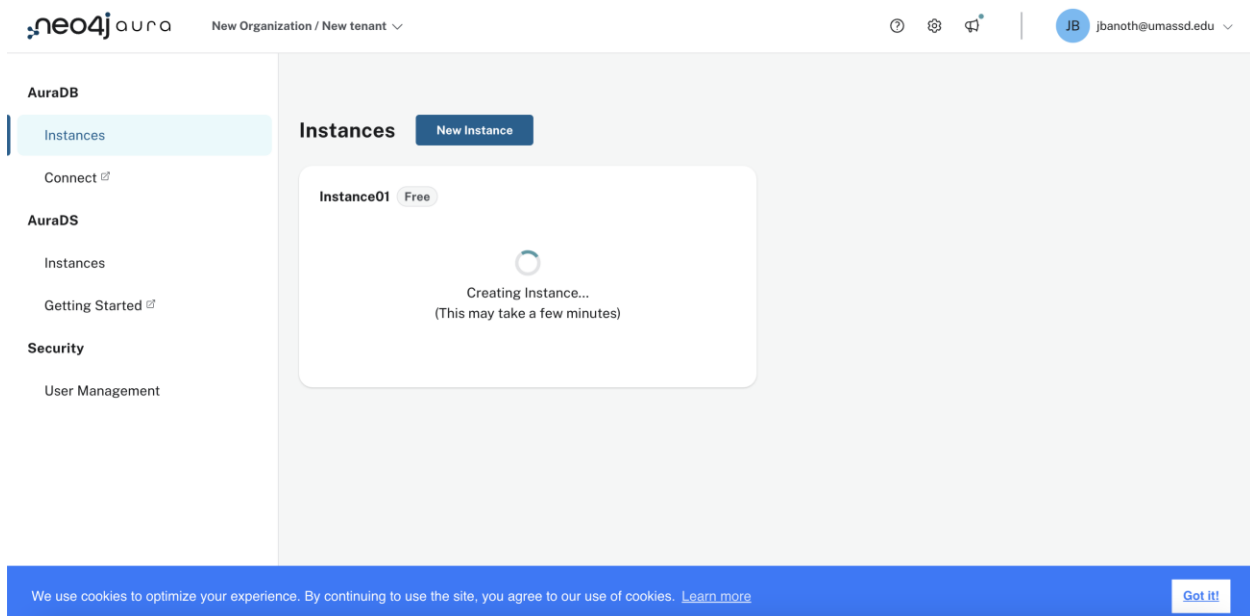
The image shows the Neo4j AURA 'Create account' page. On the left, the Neo4j AURA logo is displayed above the tagline 'Build fast, scalable, and intelligent applications in the cloud.' Below this, four features are listed with checkmarks: 'Fully-managed updates and patches', 'Robust security, reliability and ACID compliance', 'Built-in tools to learn, build, and visualize', and 'No credit card required'. On the right, a white card titled 'Create account' contains a registration form. The form asks for an 'Email address (Business preferred)*' with the example 'jbanoth@umassd.edu' entered. A 'Continue' button is below the input field. Below the button, there is a link for 'Already have an account? Log in'. Further down, separated by an 'OR' divider, are two options: 'Continue with Google' and 'Continue with Organization SSO'. At the bottom of the page, a link reads 'Trouble logging in? Please check our Article on login issues'.

By using the provided link from the lab manual, I have started creating the Neo4j account by just giving some credentials.

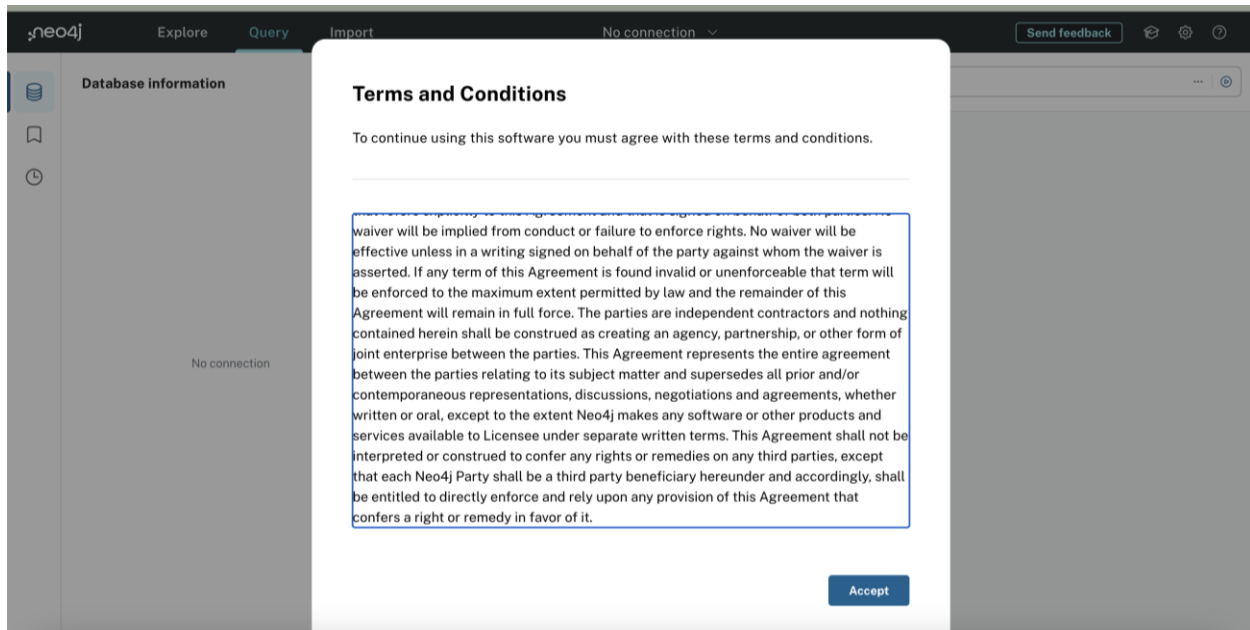


The image shows a screenshot of the Neo4j AURA web interface. A modal dialog titled 'Credentials for Instance01' is open in the center. It displays 'Username: neo4j' and a 'Generated password' field containing the string 'ITaxuQITTo8gN3KYEd2RU2wUA4dDUsk5tN-0NVKTasI'. Below the password field is a yellow warning box with a triangle icon and the text 'Note that the password will not be available after this point.' At the bottom of the dialog are two buttons: 'Close' and 'Download and continue'. The background interface shows a sidebar with navigation links for 'AuraDB' (Instances, Connect), 'AuraDS' (Instances, Getting Started), and 'Security' (User Management). The top navigation bar includes the Neo4j AURA logo, 'New Organization / New tenant', and a user profile dropdown for 'JB jbanoth@umassd.edu'. A blue footer bar at the bottom contains a cookie consent message and a 'Got it!' button.

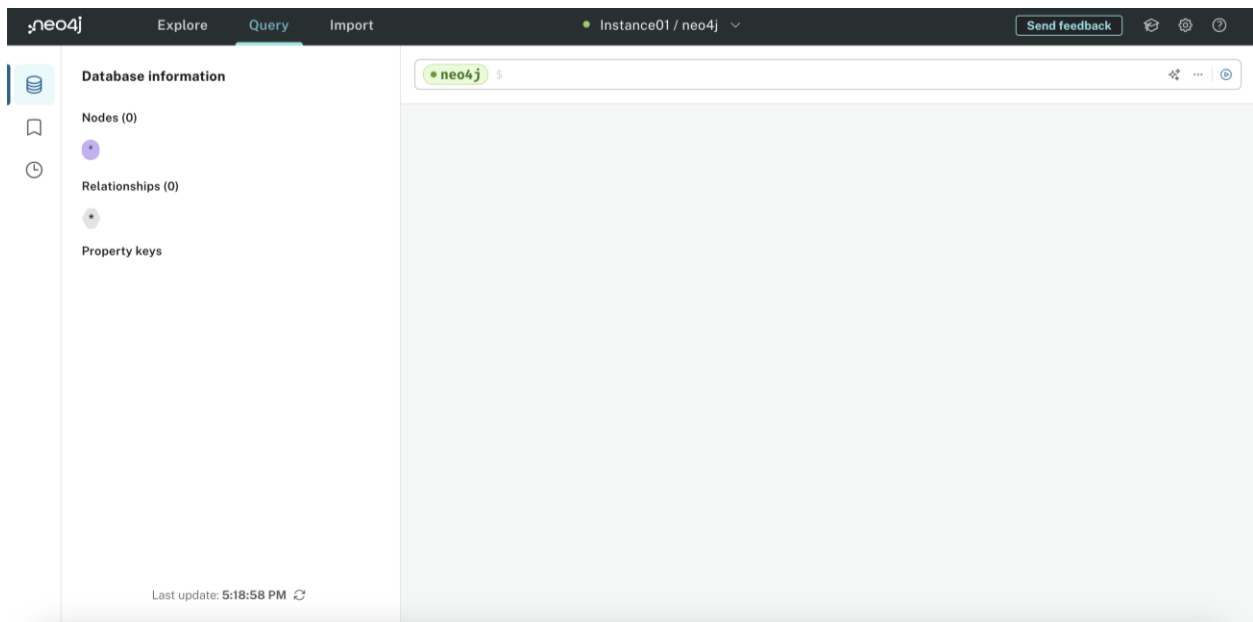
After successful creation it will ask you for free account or paid one, then I have selected the free account and proceeded further. As you can see above, I got these credentials, so I downloaded them and proceeded to the next step.



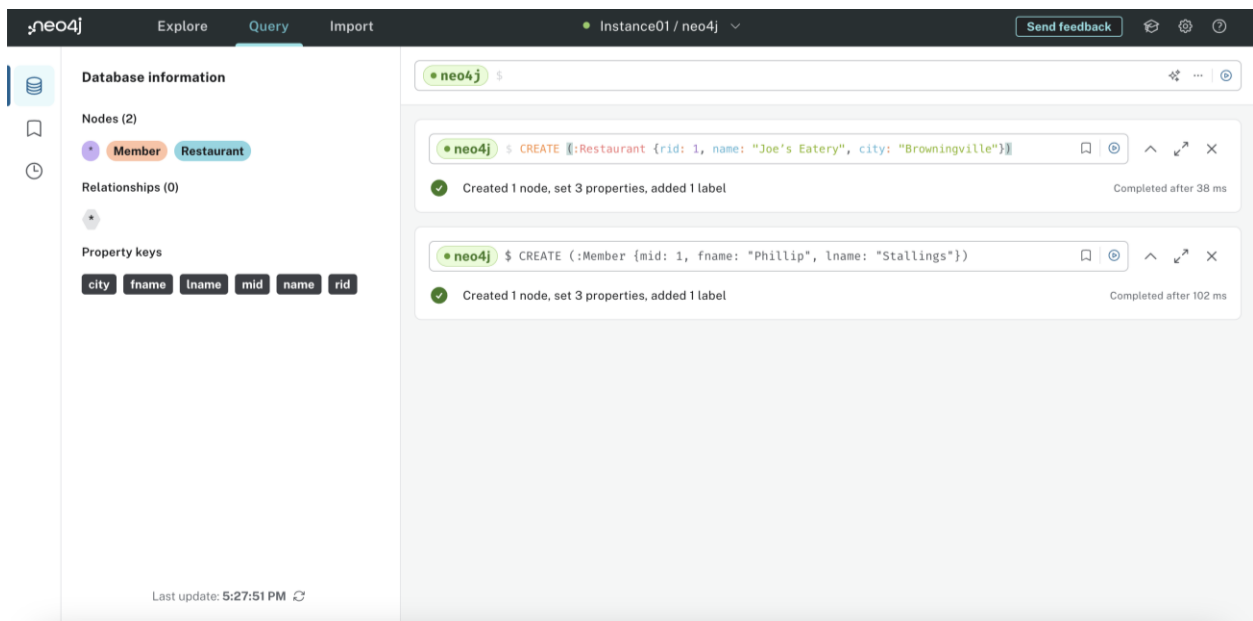
Now the instance is being created, it will be going to take few minutes for creating instance. When it is ready just press Open instance and it will be redirected to a new window.



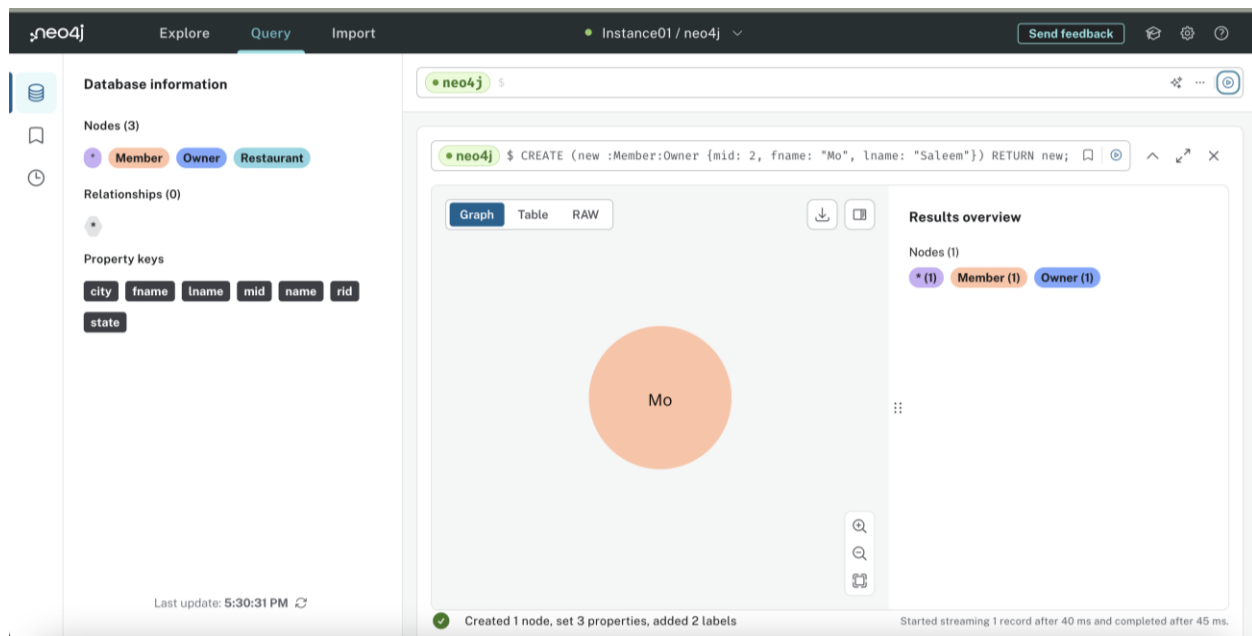
As you can see Terms and conditions will be shown in the next window, continue by accepting them.



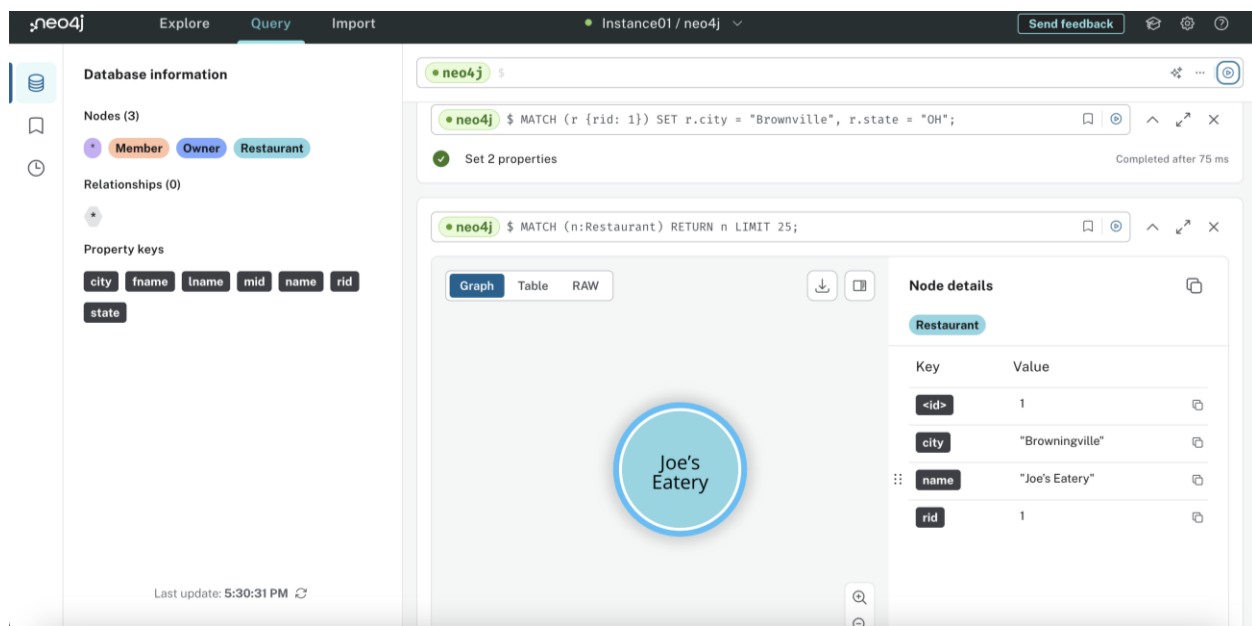
This is how the interface looks like, and you can see the username as neo4j, and we can always rename it.



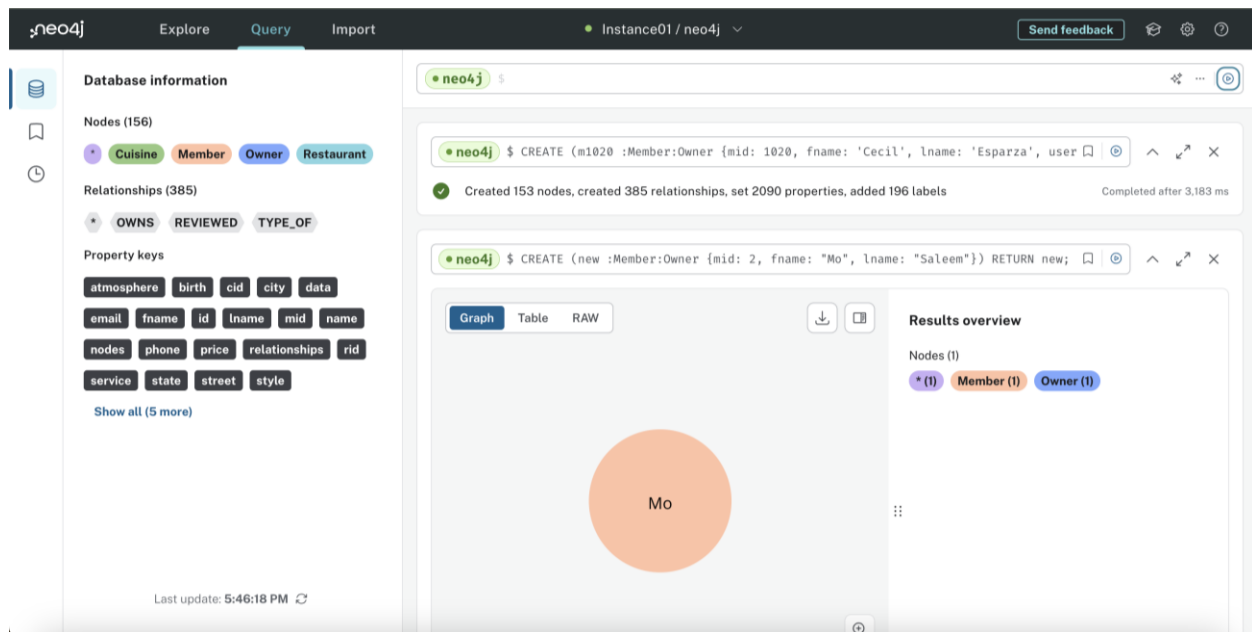
Now, we will be creating the instances by using given commands, which will create member and Restaurant as two nodes.



These are the graphical views for the member node.



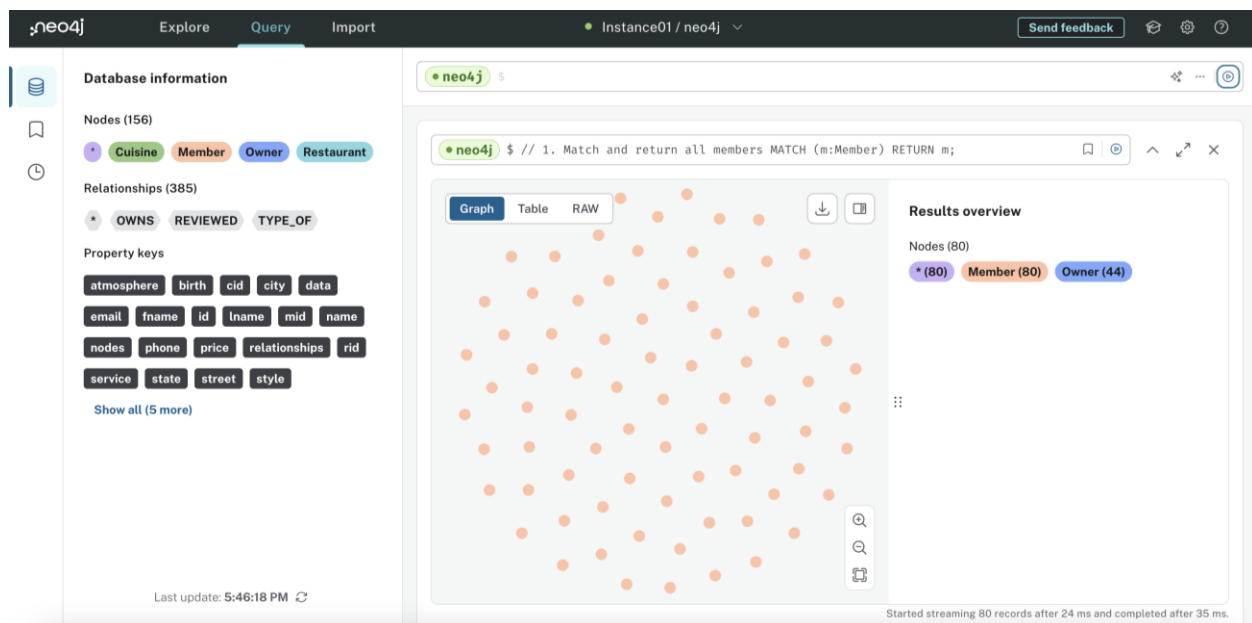
This is the graphical view of creation of Joe's Eatery node.



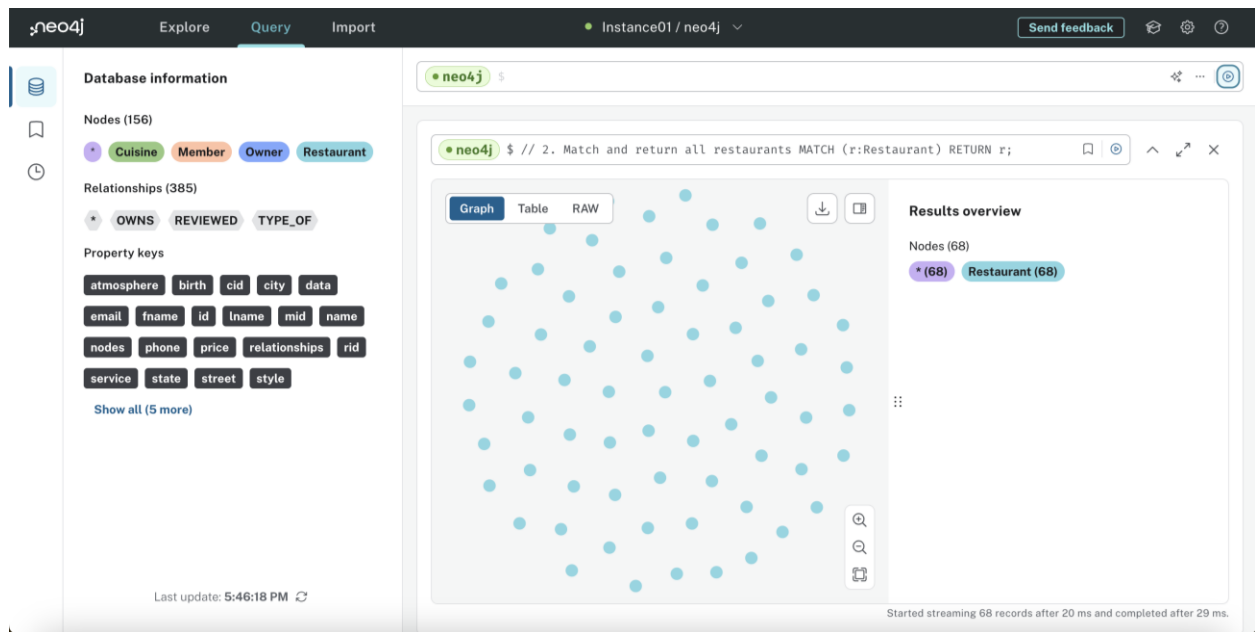
Here, we have created the relationships using the text file given in my courses. Which is we have created 153 nodes, 385 relationships, 2090 properties and 196 labels.

Queries:

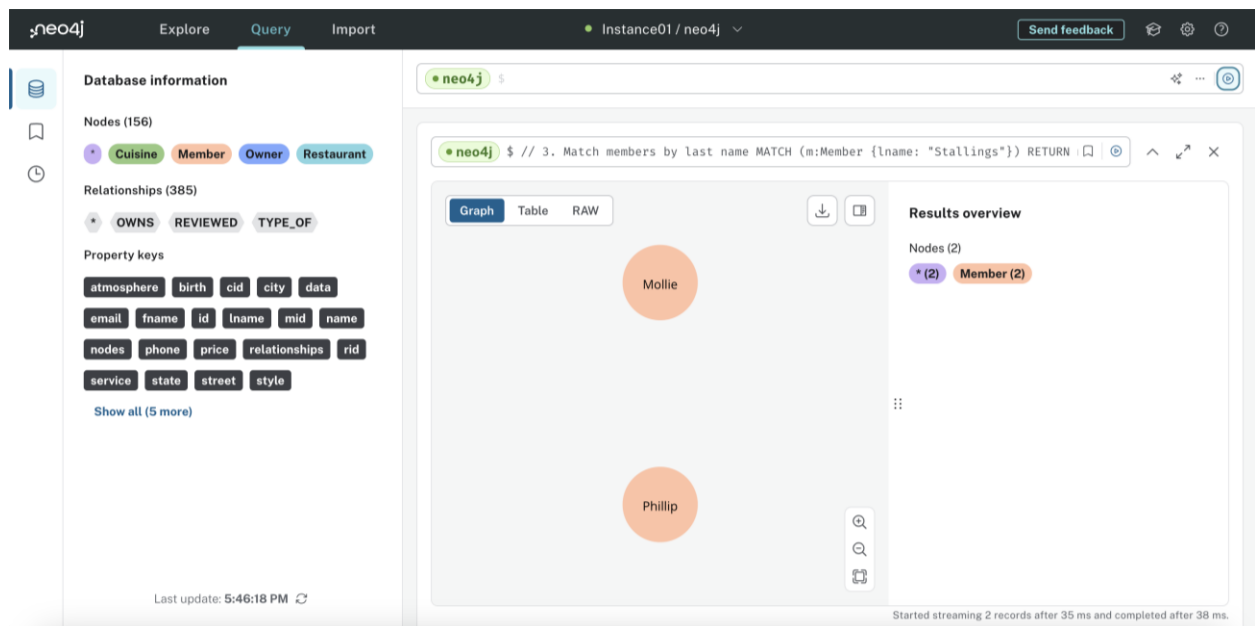
Now we will be performing queries on the database as follows:



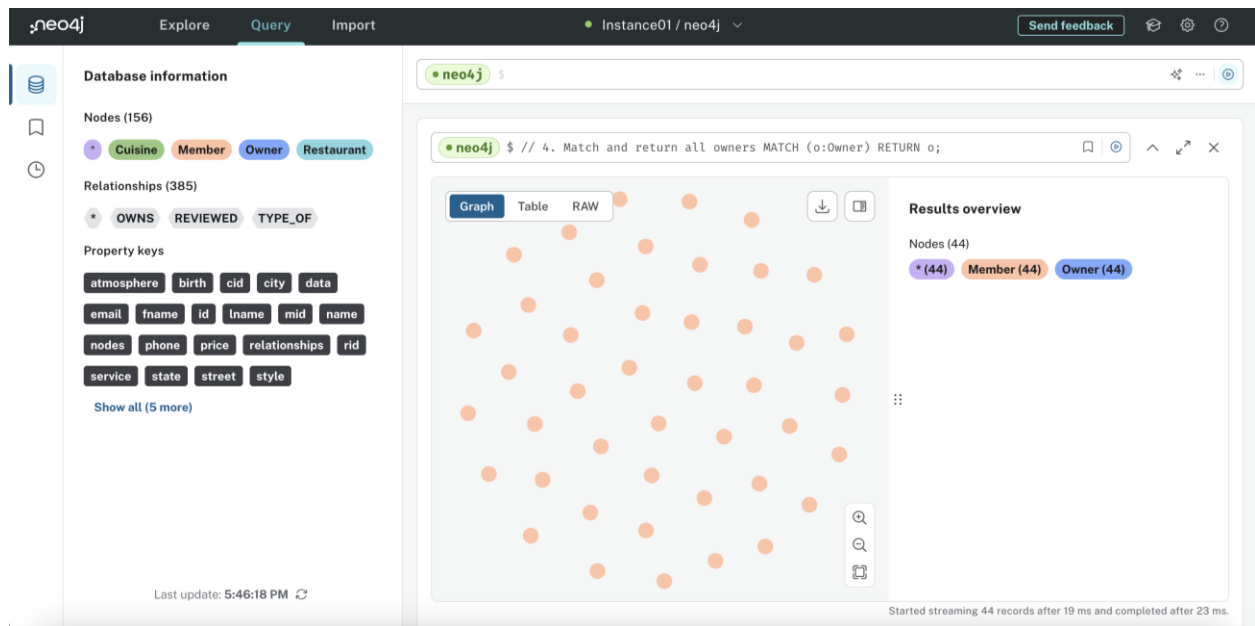
By using the above query, it will return all the members of database.



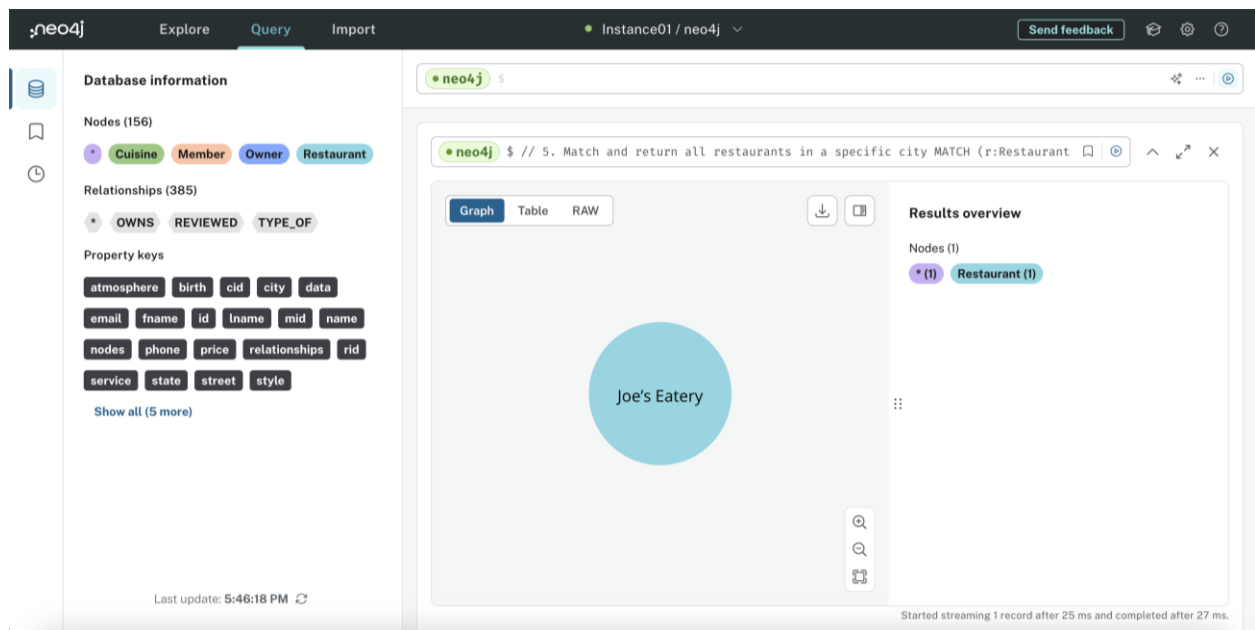
In this query, this will return all the restaurants match.



This will return match members by their last names.



This gives you all owners match.



This is the last query, and it will return all restaurants in specific city by match.

So, these are all the queries I performed and after the I deleted the instance and even account also.

Conclusion:

The first thing the lab did was go to the Neo4j website and register for an account in Neo4j Aura. The user gets a file with the login credentials for the instance after registering and creating one. The user was prepared to configure their Neo4j workspace as soon as they connected.

Using the Neo4j Browser's CREATE command, the user went on to create nodes and edges. The commands' usage of curly quotes caused problems at first. The syntax issues were fixed by changing the quotations to straight quotes, which made it possible to create nodes that represented members and restaurants. The Neo4j Browser's graphical notation capability allowed the user to visually examine these nodes and edges.

The user then adjusted the data by assigning new properties and matching nodes. This involved adding a new member who was also an owner and amending the restaurant's state and city. In addition to additional CREATE commands, MATCH and SET commands were used to complete these tasks.

After that, the user developed a brand-new database especially for a project involving food critics. The user made data input easier by copying and pasting CREATE statements from a given file to fill this database. The MATCH command was used by the user to execute multiple searches to retrieve and examine the data. These searches included city and last name-based filters, as well as matching and returning all members, eateries, and owners.

Following Neo4j Aura's instructions, the user had to destroy the created database after finishing the necessary queries and data modifications. Throughout the procedure, the user was instructed to consult the Neo4j Aura documentation for additional details on completing both simple and complex tasks. This all-encompassing strategy made sure the user could successfully complete the lab criteria and make the most of Neo4j Aura's database's capabilities.